TALARIA

TOKEN EFFICIENCY PROTOCOL

Technical documentation for the TALARIA protocol and reference web application — a token-efficiency layer for autonomous coding agents, with an on-chain verification model.

Version: 0.1.0
License: 100% free · no subscription · open source
Base layer: NousResearch / hermes-agent
Network: Base Mainnet (chain 8453)
Document type: Technical specification & architecture

1. Overview

TALARIA is a token-efficiency protocol that patches an agent's file-operation tooling to reduce token spend during iterative editing loops — the most common operation an autonomous coding agent performs. It introduces two changes to `tools/file_operations.py`:

- **Delta Reads** — when a file is re-read, only the changed hunks are sent back to the model, not the full file.
- **Compact gutter format** — line-number gutters drop leading padding (`␣34␣` → `34|`), which would otherwise tokenize into extra tokens on every line.

Together these cut up to ~40% of token spend on iterative editing loops while keeping line numbers the model can reference.

> **Why line numbers stay:** the model references gutter numbers when patching. Removing them entirely (the "no numbers" variant) caused off-by-one errors. Only the *padding* is removed, never the numbers.

2. Token Efficiency Model

2.1 Gutter format comparison

Format	Example	Token overhead vs bare
Padded (legacy)	`␣1␣␣import os`	+48%
Compact	`1| import os`	+16%
Compact + Delta	re-reads send only diffs	−24%
No numbers	`import os`	0% (but off-by-one errors)

2.2 A/B result battery

Model: Sonnet 4.6 · 2 passes · 4-task battery.

Variation	Result	Notes	Δ Tokens
padded (legacy)	4/4 PASS	baseline format	+48%
compact	4/4 PASS	numbers referenced correctly	+16%
compact + delta	4/4 PASS	re-reads send only diffs	−24%
no numbers	3/4 PASS	off-by-one, hand-counted	0%

2.3 Unaffected by the format change

- patch / fuzzy_match — match text, never consume the gutter

- no downstream parser keys on fixed-width columns
- editing behavior is identical
- blockchain hash verification is unaffected

3. Architecture

The reference web application is a single-page, tabbed dashboard built with the following stack:

Next.js 16.2.7 | React 19 | TypeScript | Tailwind CSS v4 | GSAP + @gsap/react | Turbopack

3.1 Application shell

```
src/app/
  layout.tsx        TALARIA metadata, root layout
  page.tsx          client root: tab state + animated tab transitions
  globals.css       dark theme tokens, keyframes, hover utilities

src/components/
  Header.tsx        winged-sandal logo, shimmer title, intro stagger
  Ticker.tsx        scrolling status ticker
  TabNav.tsx        tabs + sliding GSAP indicator
  StatsSection.tsx  token-efficiency hook, live counters, A/B table
  ConfigSection.tsx    format/agent/blockchain settings, raw JSON
  ChainSection.tsx     live block feed, on-chain proofs, hash verifier
  DocsSection.tsx      embedded technical documentation (this PDF)
  BackgroundFX.tsx     interactive particle constellation + cursor glow
  anim/                reusable GSAP primitives (see §4)
```

3.2 Tabs

#	Tab	Purpose
1	STATS	Token-efficiency hook, live counters, format comparison, A/B results
2	CONFIG	Format protocol, Hermes agent settings, blockchain configuration
3	CHAIN	Network status, on-chain proofs, live block feed, hash verifier
4	DOCS	This technical documentation

4. Animation System (GSAP)

All GSAP plugins are registered once in a single client-only module so they work safely with SSR:

```
// src/lib/gsap.ts
"use client";
import { gsap } from "gsap";
```

```
import { useGSAP } from "@gsap/react";
import { ScrollTrigger } from "gsap/ScrollTrigger";
import { ScrambleTextPlugin } from "gsap/ScrambleTextPlugin";
import { SplitText } from "gsap/SplitText";
gsap.registerPlugin(useGSAP, ScrollTrigger, ScrambleTextPlugin, SplitText);
export { gsap, useGSAP, ScrollTrigger, ScrambleTextPlugin, SplitText };
```

4.1 Reusable primitives

Component	Role
`Reveal`	Scroll-triggered fade/slide-in; can stagger immediate children.
`SplitReveal`	Per-character heading reveal via SplitText.
`Scramble`	ScrambleText glitch reveal (used on the newest block hash).
`CountUp`	Scroll-triggered numeric count-up with an infinite drift creep.
`Magnetic`	Cursor-following magnetic wrapper for CTAs (quickTo + contextSafe).
`BackgroundFX`	Canvas particle constellation attracted to the cursor + radial glow.

> Every animation uses the `useGSAP()` hook with a `scope`, so tweens and ScrollTriggers are automatically reverted on unmount — no manual cleanup or SSR leaks.

5. On-Chain Verification Layer

TALARIA is 100% free — there is no payment, subscription or paywall. All features (Delta Reads, compact gutter and on-chain verification) are available to everyone. The on-chain layer below exists purely to verify and audit reads, not to charge for them.

5.1 Network

Parameter	Value
Network	Base Mainnet
Chain ID	8453
RPC	mainnet.base.org
Block time	~2 seconds

5.2 On-chain verification

Every format change is hashed to Base, producing an immutable audit trail. The CHAIN tab renders a live block feed and a hash verifier UI. Verification is independent of the gutter format change.

6. Configuration Reference

```
{
  "gutter_format": "compact",
  "delta_reads": true,
  "branch_mode": "none",
  "fuzzy_match": true,
  "model": "hermes-3-70b",
  "max_tokens": 4096,
  "temperature": 0.7,
  "on_chain_verify": true,
  "network": "base-mainnet",
  "chain_id": 8453,
  "contract_addr": "0x8335...0913",
  "gas_strategy": "standard"
}
```

7. Running Locally

```
npm install
npm run dev      # starts the Next.js dev server (Turbopack)
# open https://talariaos.xyz
```

The protocol patches `tools/file_operations.py` in the host agent; the web app documented here is the dashboard and protocol front-end.
